

}

The expression following the keyword **switch** in any „C“ expression that must yield an integer value. It must be an integer constants like 1,2,3 .

The keyword **case** is followed by an integer or a character constant, each constant in each must be different from all the other.

First the integer expression following the keyword **switch** is evaluated. The value it gives is searched against the constant values that follow the **case** statements. When a match is found, the program executes the statements following the case. If no match is found with any of the case statements, then the statements following the **default** are executed.

Rules for writing switch() statement.

- 1 : The expression in switch statement must be an integer value or a character constant.
- 2 : No real numbers are used in an expression.
- 3 : The default is optional and can be placed anywhere, but usually placed at end.
- 4 : The case keyword must terminate with colon (:).
- 5 : No two case constants are identical.
- 6 : The case labels must be constants.

Valid Switch	Invalid Switch	Valid Case	Invalid Case
switch(x)	switch(f)	case 3;	case 2.5;
switch(x>y)	switch(x+2.5)	case 'a';	case x;
switch(a+b-2)		case 1+2;	case x+2;
switch(func(x,y))		case 'x'>'y';	case 1,2,3;

Example

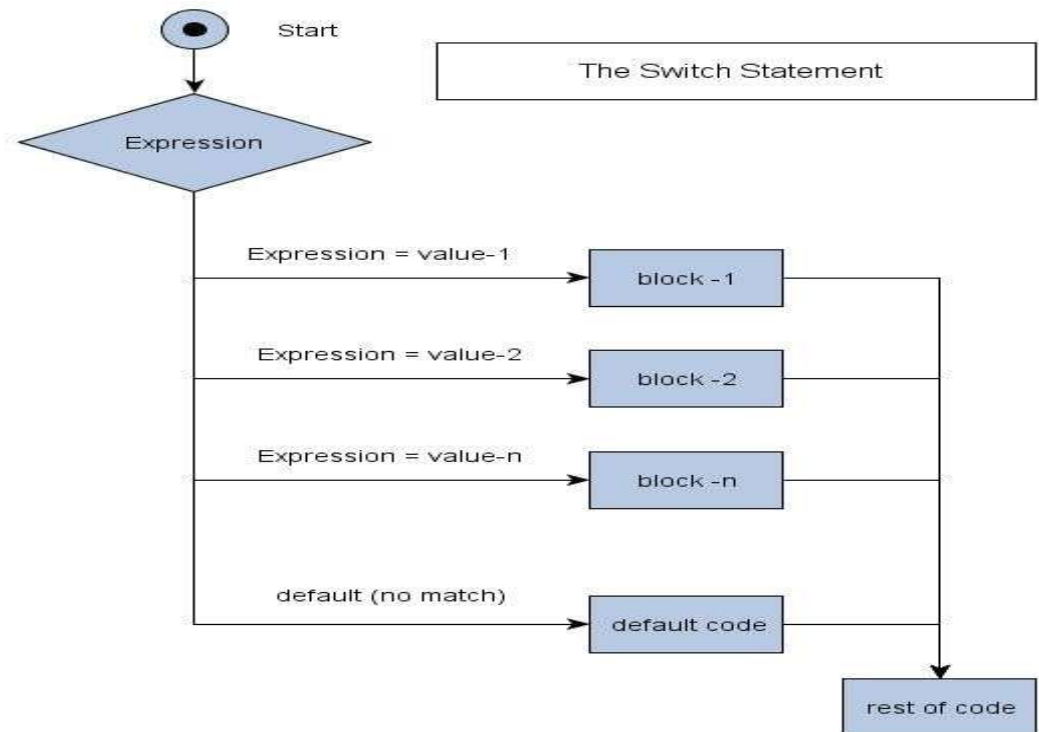
```
#include<stdio.h>
main()
{
int a;
printf("Please enter a no between 1 and 5: ");
scanf("%d",&a);
switch(a)
{
case 1:
printf("You chose One");
break;
case 2:
```

```

printf("You chose Two");
break;
case 3:
printf("You chose Three");
break;
case 4:
printf("You chose Four");
break;
case 5: printf("You chose Five.");
break;
default :
printf("Invalid Choice. Enter a no between 1 and 5"); break;
}
}

```

Flowchart



Points to Remember

It isn't necessary to use **break** after each block, but if you do not use it, all the consecutive block of codes will get executed after the matching block.

```
1. int i = 1;
2. switch(i)
3. {
4.     case 1:
5.         printf("A");           // No break
6.     case 2:
7.         printf("B");           // No break
8.     case 3:
9.         printf("C");
10.    break;
11. }
```

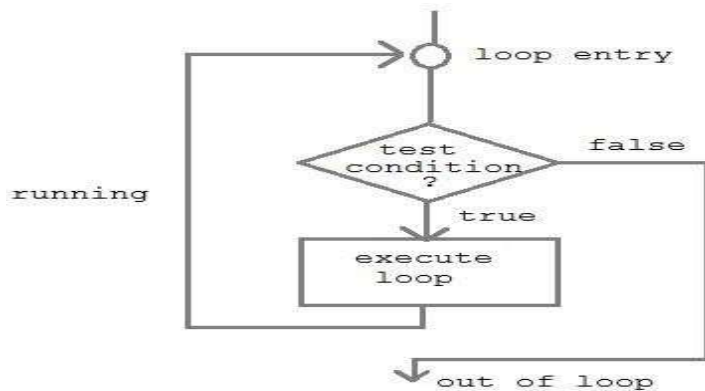
Output : A B C

The output was supposed to be only **A** because only the first case matches, but as there is no break statement after the block, the next blocks are executed, until the cursor encounters a break.

default case can be placed anywhere in the switch case. Even if we don't include the default case switch statement works.

Iteration Statements/ Loop Control Statements

How it Works



A sequence of statements are executed until a specified condition is true. This sequence of statements to be executed is kept inside the curly braces { } known as the Loop body. After every execution of loop body, condition is verified, and if it is found to be true the loop body is executed again. When the condition check returns false, the loop body is not executed.

The *loops in C language* are used to execute a block of code or a part of the program several times. In other words, it iterates/repeat a code or group of code many times.

Or Looping means a group of statements are executed repeatedly, **until some logical condition is satisfied**.

Why use loops in C language?

Suppose that you have to print table of 2, then you need to write 10 lines of code. By using the loop statement, you can do it by 2 or 3 lines of code only.

A looping process would include the following four steps.

- 1 : Initialization of a condition variable.
- 2 : Test the condition.
- 3 : Executing the body of the loop depending on the condition.
- 4 : Updating the condition variable.

C language provides three iterative/repetitive loops.

1 : while loop

2 : do-while loop

3 : for loop

While Loop: Syntax :

variable initialization ;

while (condition)

{

statements ;

variable increment or decrement ;

}

while loop can be addressed as an **entry control** loop. It is completed in 3 steps.

- Variable initialization.(e.g int x=0;)
- condition(e.g while(x<=10))
- Variable increment or decrement (x++ or x-- or x=x+2)

The while loop is an entry controlled loop statement, i.e means the condition is evaluated first and it is true, then the body of the loop is executed. After executing the body of the loop, the condition is once again evaluated and if it is true, the body is executed once again, the process of repeated execution of the loop continues until the condition finally becomes false and the control is transferred out of the loop.

Example : Program to print first 10 natural numbers

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main( )
```

```
{
```

```
int x;  
x=1;  
while(x<=10)  
{  
    printf("%d\t", x);  
    x++;  
}  
getch();  
}
```

Output

1 2 3 4 5 6 7 8 9 10

C Program to reverse number

```
#include<stdio.h>  
  
#include<conio.h>  
  
main()  
{  
    int n, reverse=0, rem;  
    clrscr();  
    printf("Enter a number: ");  
    scanf("%d", &n);
```

```
while(n!=0)
{
    rem=n%10;
    reverse=reverse*10+rem;
    n/=10;
}
printf("Reversed Number: %d",reverse);
getch();
}
```

Flowchart

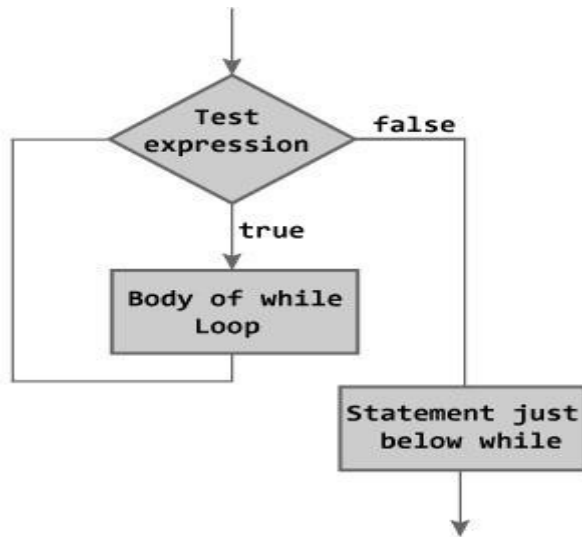


Figure: Flowchart of while Loop

do-while loop

Syntax : variable initialization ;

```
do{  
    statements ;  
    variable increment or decrement ;  
}while (condition);
```

The **do-while** loop is an **exit controlled loop statement**. The body of the loop is executed first, and then the condition is evaluated. If it is true, the body of the loop is executed once again. The process of execution of the body of the loop is continued until the condition finally becomes false, and the control is transferred to the statement immediately after the loop. The statements are always executed at least once.

Flowchart

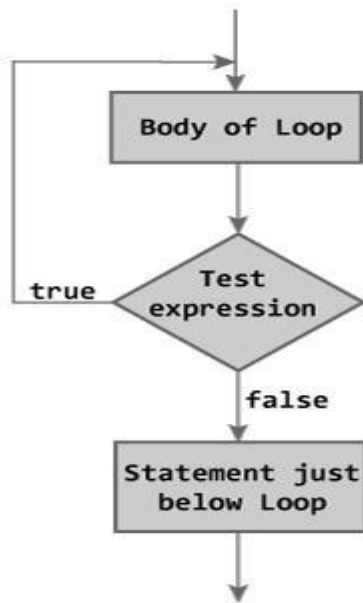


Figure: Flowchart of do...while Loop

Example : Program to print first ten multiple of 5

```
#include<stdio.h>

#include<conio.h>

void main()

{

    int a,i;

    a=5;

    i=1;

    do

    {

        printf("%d\t",a*i);

        i++;

    }while(i <= 10);

    getch();

}
```

Output

5 10 15 20 25 30 35 40 45 50

Example

```
main()
{
    int i=0
    do
    {
        printf("while vs do-while\n");
    }while(i==1);
    printf("Out of loop");
}
```

Output:

while vs do-while

Out of loop

For Loop:

- This is an **entry controlled looping** statement.
- In this loop structure, more than one variable can be initialized.
- One of the most important features of this loop is that the three actions can be taken at a time like variable initialization, condition checking and increment/decrement.
- The for loop can be more concise and flexible than that of while and do-while loops.

Syntax : for(initialization; condition; increment/decrement)

```
{
    Statements;
}
```

Example:

```
#include<stdio.h>

#include<conio.h>

void main( )

{

    int x;

    for(x=1; x<=10; x++)

    {

        printf("%d\t",x);

    }

    getch();

}
```

Output

1 2 3 4 5 6 7 8 9 10

Various forms of FOR LOOP

I am using variable num in all the below examples –

1) Here instead of num++, I'm using num=num+1 which is nothing but same as num++.

for (num=10; num<20; num=num+1)

2) Initialization part can be skipped from loop as shown below, the counter variable is declared before the loop itself.

int num=10;

for (;num<20;num++)

Must Note: Although we can skip init part but semicolon (;) before condition is must, without which you will get compilation error.

3) Like initialization, you can also skip the increment part as we did below. In this case semicolon (;) is must, after condition logic. The increment part is being done in for loop body itself.

```
for (num=10; num<20; )
```

```
{  
    //Code  
    num++;  
}
```

4) Below case is also possible, increment in body and init during declaration of counter variable.

```
int num=10;
```

```
for (;num<20;)
```

```
{  
    //Statements  
    num++;  
}
```

5) Counter can be decremented also, In the below example the variable gets decremented each time the loop runs until the condition num>10 becomes false.

```
for(num=20; num>10; num--)
```

Program to calculate the sum of first n natural numbers

```
#include <stdio.h>
```

```
int main()
```

```
{  
    int num, count, sum = 0;  
    printf("Enter a positive integer: ");  
    scanf("%d", &num);  
    // for loop terminates when n is less than count
```

```

for(count = 1; count <= num; ++count)
{
    sum += count;
}

printf("Sum = %d", sum);

return 0;
}

```

Output

Enter a positive integer: 10

Sum = 55

Factorial Program using loop

```

#include<stdio.h>

#include<conio.h>

void main(){
    int i,fact=1,number;

    clrscr();

    printf("Enter a number: ");

    scanf("%d",&number);

    for(i=1;i<=number;i++){
        fact=fact*i;
    }

    printf("Factorial of %d is: %d",number,fact);

    getch();
}

```

Output:

Enter a number: 5

Factorial of 5 is: 120

Flow Chart of *for* Loop :

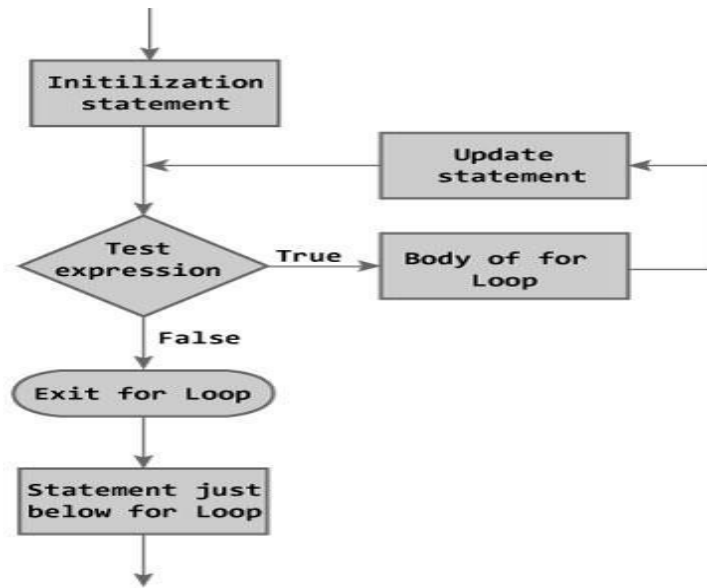
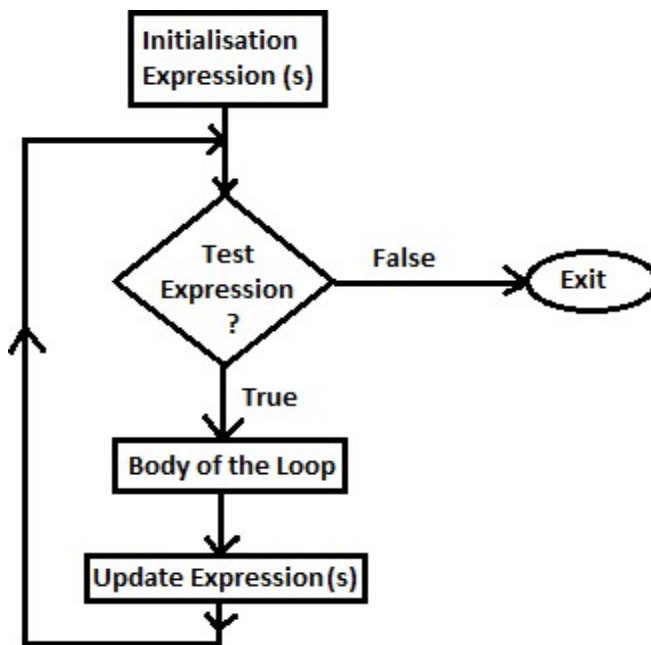


Figure: Flowchart of for Loop



Infinitive for loop in C

If you don't initialize any variable, check condition and increment or decrement variable in for loop, it is known as infinitive for loop. In other words, if you place 2 semicolons in for loop, it is known as infinitive for loop.

```
for( ; ; ){
```

```
printf("infinite for loop example by javatpoint");

}
```

Basis of Difference	For Loop	While Loop	Do While Loop
Where to Use <i>for</i> Loop, <i>while</i> Loop and <i>do while</i> Loop	The <i>for</i> loop is appropriate when we know in advance how many times the loop will be executed.	The other two loops i.e. <i>while</i> and <i>do while</i> loops are more suitable in the situations where it is not known before hand when the loop will terminate.	
		In case if the test condition fails at the beginning, and you may not want to execute the body of the loop even once if it fails, then the <i>while</i> loop should be preferred.	In case if the test condition fails at the beginning, and you may want to execute the body of the loop atleast once even in the failed condition, then the <i>do while</i> loop should be preferred.
How all the three loops works?	A <i>for</i> loop initially initiates a counter variable (initialization-expression), then it checks the test-expression, and executes the body of the loop if the test expression is true. After executing the body of the loop,	A <i>while</i> loop will always evaluate the test-expression initially. If the test-expression becomes true, then the body of the loop will be executed. The update	A <i>do while</i> loop will always executed the code in the <code>do {}</code> i.e. body of the loop block first and then evaluates the condition. In this case also, the counter variable is initialized outside the body of the loop.

	the update-expression is executed which updates the value of counter variable.	expression should be updated inside the body of the while. However, the counter variable is initialized outside the body of the loop.	
Position of the statements :	- Initialization - test-expression - update-expression	In for loop, all the three statements are placed in one position	In while and do while loop, they are placed in different position.
Syntax of Loops	for (initialization- exp.(s); test-expression(s); update- expression(s)) { body-of-the-loop ; }	while(test- expression) { body-of-the- loop; update- expression(s); }	do { body-of-the- loop; update- expression(s); } while (test- expression);

Which one is Entry Controlled Loop and Which one is Exit Controlled Loop ?	Both loops i.e. for loop and while loop are entry controlled loop, means condition is checked first and if the condition is true then the body of the loop will executes.			do while loop is an exit controlled loop, means means that condition is placed after the body of the loop and is evaluated before exiting from the loop.
Conversion of one Loop to another Loop or Example : Print numbers from 1 to 10 using all the three loops.	: : for (int i=1; i<=10; i++) { Printf(“%d”,i); }	int i = 1; : : while (i<=10) { Printf(“%d”,i); ++i }	int i = 1; : : do { Printf(“%d”,i); ++i; } while (i<=10)	

Nested for loop

We can also have nested **for** loops, i.e one **for** loop inside another **for** loop. nesting is often used for handling multidimensional arrays.

Syntax:

```
for(initialization; condition; increment/decrement)
{
    for(initialization; condition; increment/decrement)
```

```
{  
    statement ;  
}  
}
```

Example:

main()

```
{  
    for (int i=0; i<=5; i++)  
    {  
        for (int j=0; j<=5; j++)  
        {  
            printf("%d, %d",i ,j);  
        }  
    }  
}
```

Example : Program to print half Pyramid of numbers

```
#include<stdio.h>  
  
#include<conio.h>  
  
void main( )  
  
{  
  
    int i,j;  
  
    for(i=1;i<5;i++)  
  
    {  
  
        printf("\n");
```

```
for(j=i;j>0;j--)  
{  
    printf("%d",j);  
}  
}  
getch();  
}
```

Output

```
1  
21  
321  
4321  
54321
```

Jump Statements

Jumping statements are used to transfer the program's control from one location to another, these are set of keywords which are responsible to transfer program's control within the same block or from one function to another.

There are four jumping statements in C language:

- goto statement
- return statement
- break statement
- continue statement

goto statement : goto statement doesnot require any condition. This statement passes control anywhere in the program i.e, control is transferred to another part of the program without testing any condition.

Syntax : goto label;

.....

.....

label:

statements;

In this syntax, **label** is an identifier.

When, the control of program reaches to goto statement, the control of the program will jump to the **label:** and executes the code below it.

Or

The goto statement requires a label to identify the place to move the execution. A label is a valid variable/identifier name and must be ended with colon (:)

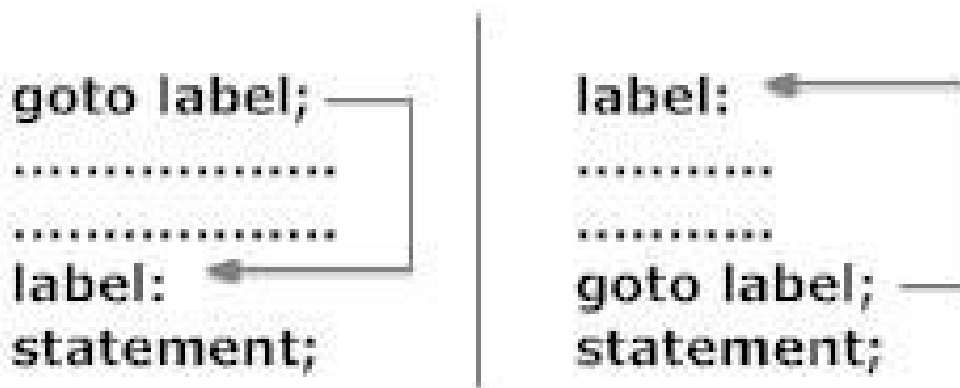
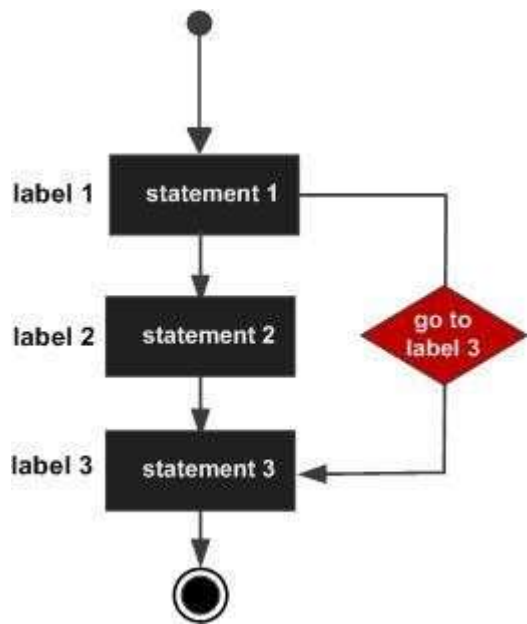


Figure: Working of goto statement

Flowchart



Example

```
int main()
{
    int age;
    Vote:
        printf("you are eligible for voting");
    NoVote:
        printf("you are not eligible to vote");
    printf("Enter you age:");
    scanf("%d", &age);
    if(age >= 18)
        goto Vote;
    else
        goto NoVote;
    return 0;
```

```
}
```

Output

Enter you age:19

you are eligible for voting

Enter you age:15

you are not eligible to vote

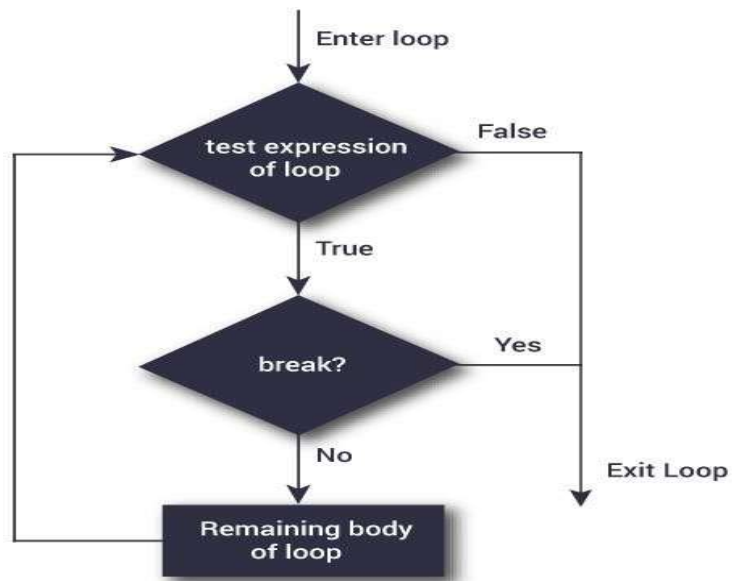
Break Statement

Break is a keyword. The break statement terminates the loop (for, while and do...while loop) immediately when it is encountered. The break statement is used/ associated with decision making statement such as if ,if-else.

Syntax of break statement

```
break;
```

Flowchart

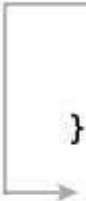


How break statement works?


```

while (test Expression)
{
    // codes
    if (condition for break)
    {
        break;
    }
    // codes
}

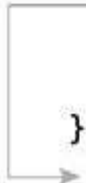
```



```

for (init, condition, update)
{
    // codes
    if (condition for break)
    {
        break;
    }
    // codes
}

```



Example

```

#include <stdio.h>

#include <conio.h>

void main(){

    int i=1;//initializing a local variable

    clrscr();

    //starting a loop from 1 to 10

    for(i=1;i<=10;i++){

```

```
printf("%d \n",i);

if(i==5){//if value of i is equal to 5, it will break the loop

break;

}

} //end of for loop

getch();

}
```

Output

12345

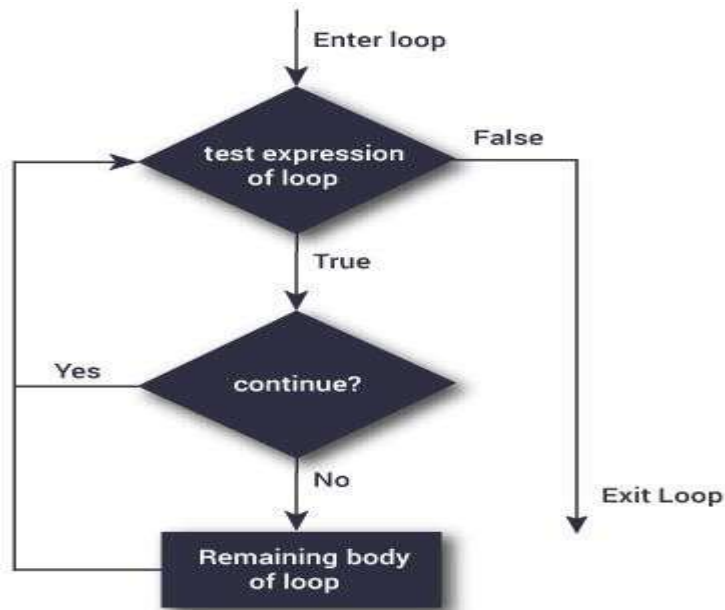
Continue Statement

Continue is keyword exactly opposite to break. The continue statement is used for continuing next iteration of loop statements. When it occurs in the loop it does not terminate, but it skips some statements inside the loop / the statements after this statement. . The continue statement is used/ associated with decision making statement such as if ,if-else.

Syntax of continue Statement

```
continue;
```

Flowchart of continue Statement



How continue statement works?

```
→ while (test Expression)
{
    // codes
    if (condition for continue)
    {
        continue;
    }
    // codes
}
```

```
→ for (init, condition, update)
{
    // codes
    if (condition for continue)
    {
        continue;
    }
    // codes
}
```

Example

```

1. #include <stdio.h>
2. #include <conio.h>
3. void main(){
4. int i=1;//initializing a local variable
5. clrscr();
6. //starting a loop from 1 to 10
7. for(i=1;i<=10;i++){
8. if(i==5){//if value of i is equal to 5, it will continue the loop
9. continue;
10. }
11. printf("%d \n",i);
12. }//end of for loop
13. getch();
14. }

```

Output

1234678910

Comparison between break and continue statements

Break	Continue
1 : break statement takes the control to the outside of the loop	1 :continue statement takes the control to the beginning of the loop..
2 : it is also used in switch statement.	2 : This can be used only in loop statements.
3 : Always associated with if condition in loops.	3 : This is also associated with if condition.

ARRAYS

Using Arrays in C

C supports a derived data type known as *array* that can be used to handle large amounts of data (multiple values) at a time.

Definition:

An array is a group (or collection) of same data types.

Or

An array is a collection of data that holds fixed number of values of same type.

Or

Array is a *collection* or *group* of elements (data). All the elements of array are *homogeneous* (similar). **It has contiguous memory location.**

Or

An array is a data structured that can store a fixed size sequential collection of elements of same data type.

What's the need of an array?

Suppose you have to store marks of 50 students, one way to do this is allotting 50 variables. So it will be typical and hard to manage. For example we can not access the value of these variables with only 1 or 2 lines of code.

Another way to do this is array. By using array, we can access the elements easily. Only few lines of code is required to access the elements of array.

Where arrays are used

- to store list of Employee or Student names,
- to store marks of a students,
- or to store list of numbers or characters etc.

Advantage of C Array

1) Code Optimization: Less code to the access the data.

2) Easy to traverse data: By using the for loop, we can retrieve the elements of an array easily.

3) Easy to sort data: To sort the elements of array, we need a few lines of code only.

4) Random Access: We can access any element randomly using the array.

Disadvantage of Array

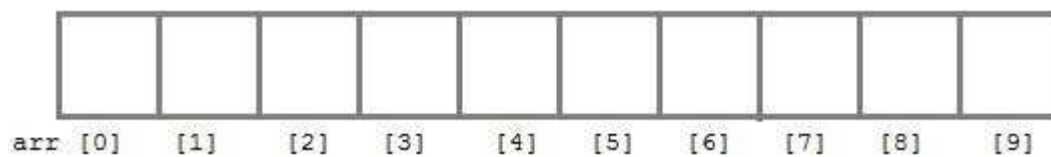
Fixed Size: Whatever size, we define at the time of declaration of array, we can't exceed the limit. So, it doesn't grow the size dynamically like LinkedList

Declaration of an Array

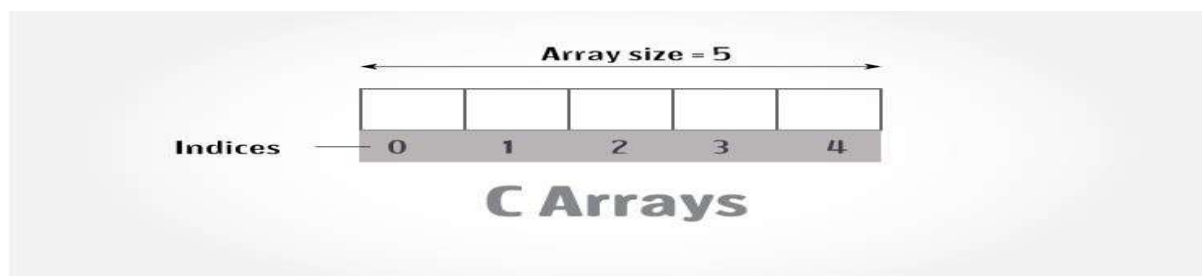
data-type variable-name[size/length of array];

For example:

```
int arr[10];
```



```
int arr[ 5];
```



Here **int** is the data type, **arr** is the name of the array and 10 is the size of array. It means array **arr** can only contain 10 elements of **int** type. **Index** of an array starts from 0 to size-1 i.e first element of **arr** array will be stored at arr[0] address and last element will occupy arr[9].

Initialization of an Array

After an array is declared it must be initialized. Otherwise, it will contain **garbage** value(any random value). An array can be initialized at either **compile time** or at **runtime**.

Compile time Array initialization

Compile time initialization of array elements is same as ordinary variable initialization.

Syntax : `data_type array_name[size]={v1,v2,...vn/list of values ;`

Example

```
int age[5]={22,25,30,32,35};
```

	0	1	2	3	4
age	22	25	30	32	35

```
int marks[4]={ 67, 87, 56, 77 }; //integer array initialization
```

```
float area[5]={ 23.4, 6.8, 5.5 }; //float array initialization
```

```
int marks[4]={ 67, 87, 56, 77, 59 }; //Compile time error
```

Different ways of initializing arrays :

- 1 : Initializing all specified memory locations
- 2 : Partial array initialization.
- 3 : Initialization without size.
- 4 : String initialization.

1 : Initializing all specified memory locations : If the number of values to be initialized is equal to size of array. Arrays can be initialized at the time of declaration. Array elements can be initialized with data items of type int,float,char, etc.

Ex : consider integer initialization

```
int a[5]={10,20,30,40,50};
```

During compilation, 5 contiguous memory locations are reserved by the compiler for the variable a and all these locations are initialized.

The array a is initialized as

a[0]	a[1]	a[2]	a[3]	a[4]
10	20	30	40	50
1000	1002	1004	1006	1008

If the size of integer is 2 bytes, 10 bytes will be allocated for the variable a.

Ex : consider character initialization

```
char b[8] = {„C“, „O“, „M“, „P“, „U“, „T“, „E“, „R“};
```

The array b is initialized as

b[0]	b[1]	b[2]	b[3]	b[4]	b[5]	b[6]	b[7]
C	O	M	P	U	T	E	R

Other Examples : `char b[5]={„J“, „B“, „R“, „E“, „C“, „B“};`

//error : number of initial values are more than the size of array.

Other Example : `int a[5]={10,20,30,40,50,60};`

//error : Number of initial values are more than the size of array.

2 : Partial Array Initialization : partial array initialization is possible in C language. If the number of values to be initialized is less than the size of the array, then the elements are initialized in the order from 0th location. The remaining locations will be initialized to zero automatically.

Ex : Consider the partial initialization

```
int a[5]={10,15};
```

Eventhough compiler allocates 5 memory locations, using this declaration statement, the compiler initializes first two locations with 10 and 15, the next set of memory locations are automatically initialized to zero.

The array a is partial initialization as

a[0]	a[1]	a[2]	a[3]	a[4]
10	15	0	0	0
1000	1002	1004	1006	1008

How to access the elements of an array?

You can access elements of an array by **indices/index**. You can use array subscript (or index) to access any element stored in array. Subscript starts with 0, which means array_name[0] would be used to access first element in an array.

In general array_name[n-1] can be used to access nth element of an array. where n is any integer number.

Example

```
float mark[5];
```

Suppose you declared an array mark as above. The first element is mark[0], second element is mark[1] and so on.

mark[0] mark[1] mark[2] mark[3] mark[4]

--	--	--	--	--

Few key notes:

- Arrays have 0 as the first index not 1. In this example, mark[0]
- If the size of an array is n, to access the last element, (n-1) index is used. In this example, mark[4]
- Suppose the starting address of mark[0] is 2120d. Then, the next address, a[1], will be 2124d, address of a[2] will be 2128d and so on. It's because the size of a float is 4 bytes.

Input data into array

As you can see, in above example that I have used „for loop“ and „scanf statement“ to enter data into array. You can use any loop for data input.

Code:

```
for (x=0; x<=19;x++)
{
    printf("enter the integer number %d\n", x);
    scanf("%d", &num[x]);
}
```

Reading out data from an array

For example you want to read and display array elements, you can do it just by using any loop. Suppose array is mydata[20].

```
for (int i=0; i<20; i++)
{
    printf("%d\n", mydata[i]);
}
```

Exmaple

```
#include<stdio.h>

#include<conio.h>

void main()

{

    int i;

    int arr[]={2,3,4};    //Compile time array initialization

    for(i=0 ; i<3 ; i++) {

        printf("%d\t",arr[i]);

    }

    getch();

}
```

Output

2 3 4

Exmaple

1. include <stdio.h>
2. #include <conio.h>
3. **void** main(){
4. **int** i=0;
5. **int** marks[5]={20,30,40,50,60};*//declaration and initialization of array*
6. clrscr();
- 7.
8. *//traversal of array*
9. **for**(i=0;i<5;i++){
10. printf("%d \n",marks[i]);
11. }
- 12.
13. getch();
14. }

Output

20
30
40
50
60

Runtime Array initialization

An array can also be initialized at runtime using scanf() function. This approach is usually used for initializing large array, or to initialize array with user specified values.

Example

```
#include<stdio.h>

#include<conio.h>

void main()

{

    int arr[4];

    int i, j;

    printf("Enter array element");

    for(i=0;i<4;i++)

    {

        scanf("%d",&arr[i]);    //Run time array initialization

    }

    for(j=0;j<4;j++)

    {

        printf("%d\n",arr[j]);

    }

    getch();
```

}

Two-Dimensional Arrays

The two dimensional array in C language is represented in the form of rows and columns, also known as matrix. It is also known as *array of arrays* or *list of arrays*.

The two dimensional, three dimensional or other dimensional arrays are also known as *multidimensional* arrays.

Declaration of two dimensional Array

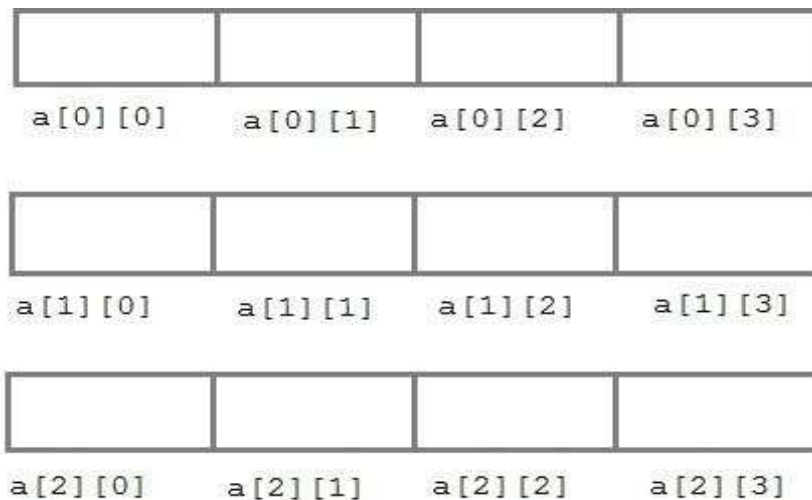
```
data_type array_name[size1][size2];
```

Example

```
int twodimen[4][3];
```

Example :

```
int a[3][4];
```



Initialization of 2D Array

```
int arr[4][3]={{1,2,3},{2,3,4},{3,4,5},{4,5,6}};
```

Accessing Two-Dimensional Array Elements

An element in a two-dimensional array is accessed by using the subscripts, i.e., row index and column index of the array.

Example

```
1. #include <stdio.h>
2. #include <conio.h>
3. void main(){
4. int i=0,j=0;
5. int arr[4][3]={{1,2,3},{2,3,4},{3,4,5},{4,5,6}};
6. clrscr();
7. //traversing 2D array
8. for(i=0;i<4;i++){
9.   for(j=0;j<3;j++){
10.    printf("arr[%d] [%d] = %d \n",i,j,arr[i][j]);
11.   }//end of j
12. }//end of i
13. getch();
14. }
```

Output

```
arr[0][0] = 1
arr[0][1] = 2
arr[0][2] = 3
arr[1][0] = 2
arr[1][1] = 3
arr[1][2] = 4
arr[2][0] = 3
arr[2][1] = 4
arr[2][2] = 5
arr[3][0] = 4
arr[3][1] = 5
arr[3][2] = 6
```

Example Write a C program Addition of Two Matrices

```
#include<stdio.h>
```

```
#include<conio.h>
```

```

void main()

{
    int a[25][25],b[25][25],c[25][25],i,j,m,n;

    clrscr();

    printf("enter the rows and columns of two matrices:\n");

    scanf("%d%d",&m,&n);

    printf("\nenter the elements of A matrices");

    for(i=0;i<m;i++)
    {
        for(j=0;j<n;j++)

            scanf("%d",&a[i][j]);
    }

    printf("\nenter the elements of B matrices");

    for(i=0;i<m;i++)
    {
        for(j=0;j<n;j++)

            scanf("%d",&b[i][j]);
    }

    printf("\nThe elements of A matrices");

    for(i=0;i<m;i++)
    {

        printf("\n");

        for(j=0;j<n;j++)

```

```

    printf("\t%d",a[i][j]);

}

printf("\n\nThe elements of B matrices");

for(i=0;i<m;i++)

{

    printf("\n");

    for(j=0;j<n;j++)

        printf("\t%d",a[i][j]);

}

printf("\n\nThe additon of two matrices");

for(i=0;i<m;i++)

{

    printf("\n");

    for(j=0;j<n;j++)

    {

        c[i][j]=a[i][j]+b[i][j];

        printf("\t%d",c[i][j]);

    }

}

getch();

}

```


Write a C program Multiplication of Two Matrices.

```
#include<stdio.h>

#include<conio.h>

void main()

{

    int a[25][25],b[25][25],c[25][25],i,j,m,n,k,r,s;

    clrscr();

    printf("enter the rows and columns of A matrices:\n");

    scanf("%d%d",&m,&n);

    printf("enter the rows and columns of B matrices:\n");

    scanf("%d%d",&r,&s);

    printf("\nenter the elements of A matrices");

    for(i=0;i<m;i++)

    {

        for(j=0;j<n;j++)

            scanf("\t%d",&a[i][j]);

    }

    printf("\nenter the elements of B matrices");

    for(i=0;i<m;i++)

    {

        for(j=0;j<n;j++)

            scanf("\t%d",&b[i][j]);
```

```
}

printf("\nThe elements of A matrices");

for(i=0;i<m;i++)

{

    printf("\n");

    for(j=0;j<n;j++)

        printf("\t%d",a[i][j]);

}

printf("\nThe elements of B matrices");

for(i=0;i<m;i++)

{

    printf("\n");

    for(j=0;j<n;j++)

        printf("\t%d",b[i][j]);

}


for(i=0;i<m;i++)

{

    printf("\n");

    for(j=0;j<n;j++)

    {

        c[i][j]=0;

        for(k=0;k<m;k++)
```

```

        c[i][j]=c[i][j]+a[i][k]*b[k][j];
    }
}

printf("\nThe Multiplication of two matrices");

for(i=0;i<m;i++)
{
    printf("\n");
    for(j=0;j<n;j++)
        printf("\t%d",c[i][j]);

    getch();
}

```

Multidimensional Arrays

How to initialize a multidimensional array?

Initialization of a three dimensional array.

You can initialize a three dimensional array in a similar way like a two dimensional array. Here's an example

```

int test[2][3][4] = {
    { {3, 4, 2, 3}, {0, -3, 9, 11}, {23, 12, 23, 2} },
    { {13, 4, 56, 3}, {5, 9, 3, 5}, {3, 1, 4, 9} }
};

```

Example

```

#include <stdio.h>

int main()
{
    // this array can store 12 elements
    int i, j, k, test[2][3][2];
    printf("Enter 12 values: \n");
    for(i = 0; i < 2; ++i) {
        for (j = 0; j < 3; ++j) {
            for(k = 0; k < 2; ++k ) {
                scanf("%d", &test[i][j][k]);
            }
        }
    }

    // Displaying values with proper index.
    printf("\nDisplaying values:\n");
    for(i = 0; i < 2; ++i) {
        for (j = 0; j < 3; ++j) {
            for(k = 0; k < 2; ++k ) {
                printf("test[%d][%d][%d] = %d\n", i, j, k, test[i][j][k]);
            }
        }
    }

    return 0;
}

```

Output

Enter 12 values:

123456789101112

Displaying Values:

test[0][0][0] = 1

test[0][0][1] = 2

test[0][1][0] = 3

test[0][1][1] = 4

test[0][2][0] = 5

test[0][2][1] = 6

test[1][0][0] = 7

test[1][0][1] = 8

test[1][1][0] = 9

test[1][1][1] = 10

test[1][2][0] = 11

test[1][2][1] = 12

STRINGS:

String Concepts

String is an *array of characters* that is terminated by \0 (null character). This null character indicates the end of the string. Strings are always enclosed by double quotes (" "). Whereas, character is enclosed by single quotes.

Or

In „C“ language the group of characters, digits, and symbols enclosed within double quotation (" ") marks are called as string otherwise a string is an array of characters and terminated by NULL character which is denoted by the escape sequence „\0“.

C Strings

Declaration of String: C does not support string as a data type. However, it allows us to represent strings as character arrays. In C, a string variable is any valid C variable name and it is always declared as an array of characters.

The general form of declaration of a string variable is :

Syntax: `char string_name[size];`

The size determines the number of characters in the string name.

Note: In declaration of string size must be required to mention otherwise it gives an error.

Ex: `char str[]; // Invalid`

`char str[0]; // Invalid`

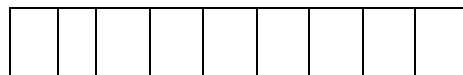
`char str[-1]; // Invalid`

`char str[10]; // Valid`

`char a[9]; //Valid`

Using this declaration the compiler allocates 9 memory locations for the variable a ranging from 0 to 8.

0 1 2 3 4 5 6 7 8



Here, the string variable a can hold maximum of 9 characters including NULL(\0) character.

Initializing Array string

Syntax : `char string_name[size]={“string” };`

Note: In Initialization of the string if the specific number of character is not initialized it then rest of all character will be initialized with NULL.

```
char str[5]={'5','+','A'};
```

```
str[0]; ---> 5
```

```
str[1]; ---> +
```

```
str[2]; ---> A
```

```
str[3]; ---> NULL
```

```
str[4]; ---> NULL
```

Note: In initialization of the string we can not initialize more than size of string elements.

Ex:

```
char str[2]={'5','+','A','B'}; // Invalid
```

Different ways of initialization can be done in various ways :

1 : Initializing locations character by character.

2 : Partial array initialization.

3 : Initialization without size.

4 : Array initialization with a string .

1 : Initializing locations character by character

Consider the following declaration with initialization,

```
Char b[9]={','C','O','M','P','U','T','E','R'};
```

The compiler allocates 9 memory locations ranging from 0 to 8 and these locations are initialized with the characters in the order specified. The remaining locations are automatically initialized to null characters.

C	O	M	P	U	T	E	R	\0
0	1	2	3	4	5	6	7	8

2 : Partial Array Initialization : If the characters to be initialized is less than the size of the array, then the characters are stored sequentially from left to right. The remaining locations will be initialized to NULL characters automatically.

Ex : Consider the partial initialization

```
int a[10]={„R“,„A“,„M“,„A“ };
```

The compiler allocates 10 bytes for the variable a ranging from 0 to 9 and initializes first four locations with the ASCII characters of „R“, „A“, „M“, „A“. The remaining locations are automatically filled with NULL characters (i.e.,\0).

R	A	M	A	\0	\0	\0	\0	\0	\0
0	1	2	3	4	5	6	7	8	9

3 : Initialization without size : consider the declaration along with the initialization

```
char b[]={„C“,„O“,„M“,„P“,„U“,„T“,„E“,„R“};
```

In this declaration, The compiler will set the array size to the total number of initial values i.e 8. The character will be stored in these memory locations in the order specified.

b[0] b[1] b[2] b[3] b[4] b[5] b[6] b[7]

C	O	M	P	U	T	E	R
---	---	---	---	---	---	---	---

4) Array Initialization with a String : consider the declaration with string initialization.

```
char b[ ] = “COMPUTER”;
```

Here, the string length is 8 bytes. But , string size is 9 bytes. So the compiler reserves 8+1 memory locations and these locations are initialized with the characters in the order specified. The string is terminated by \0 by the compiler.

C	O	M	P	U	T	E	R	\0
0	1	2	3	4	5	6	7	8

The string “COMPUTER” contin 8 charactes, because it is a string. It always ends with null character. So, the array is 9 bytes (i.e string length+1 byte for null character).

Reading and Writing Strings : The „%s“ control string can be used in scanf() statement to read a string from the terminal and the same may be used to write string to the terminal in printf() statement.

Example : char name[10];

```
scanf(“%s”,name);
```

```
printf(“%s”,name);
```

Example:

1. #include <stdio.h>
2. void main ()
3. {
4. char ch[13]={ 'c', 'p', 'r', 'o', 'g', 'r', 'a', 'm', 'm', 'i', 'n', 'g', '\0'};
5. char ch2[13]="cprogramming";
- 6.
7. printf("Char Array Value is: %s\n", ch);
8. printf("String Literal Value is: %s\n", ch2);
9. }

Output

Char Array Value is: cprogramming

String Literal Value is: cprogramming

Example:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
char name[20];
```

```
printf("Enter name: ");
```

```
scanf("%s", name);
```

```

printf("Your name is %s.", name);

return 0;

}

```

Output

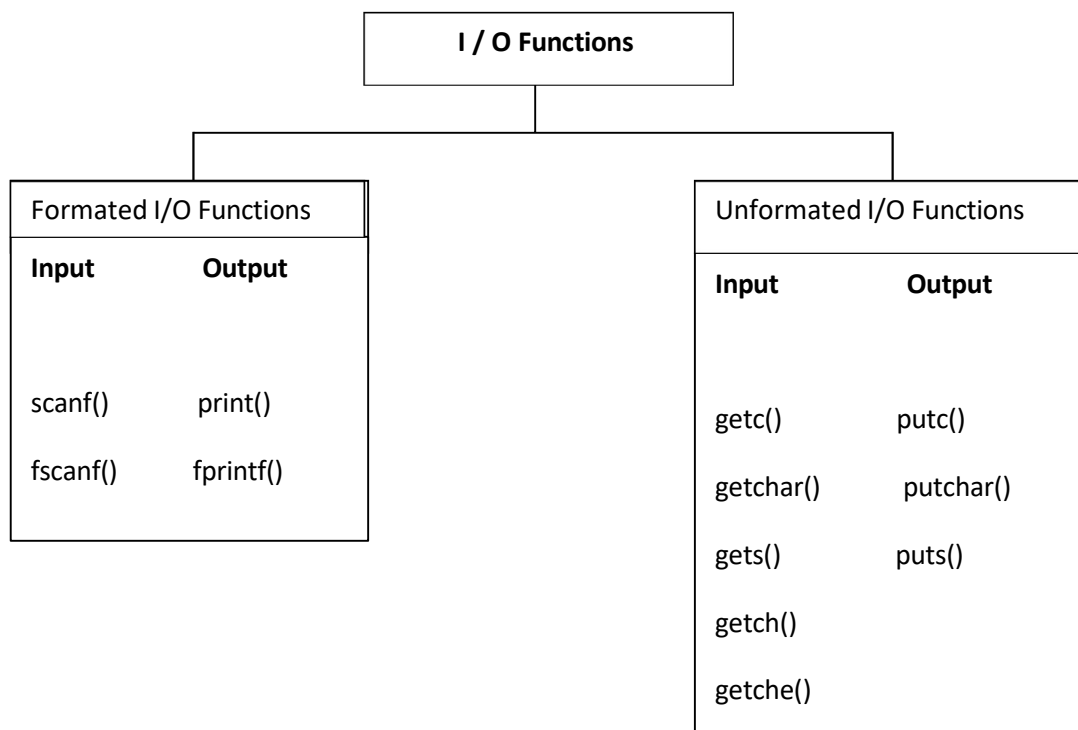
Enter name: Dennis Ritchie

Your name is Dennis.

String Input/output Functions

The strings can be read from the keyboard and can be displayed onto the monitor using various functions.

The various input and output functions that are associated with can be classified as



Unformatted I/O Functions

1 : getchar() function : A single character can be given to the computer using „C“ input library function getchar().

Syntax : char variable=getchar();

The getchar() function is written in standard I/O library. It reads a single character from a standard input device. This function does not require any arguments, through a pair of parentheses, must follow the statements getchar().

```
#include<stdio.h>
#include<conio.h>
#include<ctype.h>
void main()
{
    char ch;
    clrscr();
    printf("Enter any character/digit:");
    ch=getchar();
    if(isalpha(ch)>0)
        printf("it is a alphabet:%c\n",ch);
    else if(isdigit(ch)>0)
        printf("it is a digit:%c\n",ch);
    else
        printf("it is a alphanumeric:%c\n",ch);
    getch();
}
```

```
}.  

```

OUTPUT : Enter any character/Digit : abc

it is a alphabet:a

2 : putchar() function : The putchar() function is used to display one character at a time on the standard output device. This function does the reverse operation of the single character input function.

Syntax : putchar(character variable);

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<ctype.h>
```

```
void main()
```

```
{
```

```
    char ch;
```

```
    printf("Enter any alphabet either in lower or uppercase:");
```

```
    ch=getchar();
```

```
    if(islower(ch))
```

```
        putchar(toupper(ch));
```

```
    else
```

```
        putchar(tolower(ch));
```

```
    getch();
```

```
}
```

OUTPUT : Enter any alphabet either in lower or uppercase :a

A

3 : gets() : The gets() function is used to read the string (String is a group of characters) from the standard input device (keyboard).

Syntax : gets(char type of array variable);

Ex :#include<stdio.h>

#include<conio.h>

void main()

{

char str[40];

clrscr();

printf("Enter String name:");

gets(str);

printf("Print the string name%s:",str);

getch();

}

OUTPUT : Enter the string : reddy

Print the string :reddy

4 : puts() :The puts() function is used to display the string to the standard output device (Monitor).

Syntax : puts(char type of array variable);

Program using gets() function and puts() function.

#include<stdio.h>

#include<conio.h>

void main()

{

char str[40];

```
puts("Enter String name:");  
gets(str);  
puts("Print the string name:");  
puts(str);  
getch();  
}
```

OUTPUT :Enter string name :

subbareddy

Print the string name

subbareddy

getch() function :The getch function reads a single character directly from the keyboard, without echoing to the screen.

Syntax : int getch();

Ex : #include<stdio.h>

```
void main()  
{  
    char c;  
    c=getch();  
}
```

getche() function :The getche() function reads a single character from the keyboard and echoes it to the current text window.

Syntax : int getche();

Ex : #include<stdio.h>

```
void main()  
{
```

```

char c;

c=getche();

}

```

getc() function : This function is used to accept a single character from the standard input to a character variable.

Syntax : character variable=getc();

putc() function : This function is used to display a single character in a character variable to standard output device.

Syntax : putc(character variable);

Array of Strings

String Manipulation Functions/ String Handling Functions

The various string handling functions that are supported in C language are as shown

String Function	Description
strlen(str)	Returns the length of the string str.
strcpy(str1,str2)	Copies the string str2 to string str1
strcat(str1,str2)	Append string str2 to string str1.
strlwr(str)	Converts the string str to lowercase
strupr(str)	Converts the string str to uppercase.
strrev(str)	Reverse the string str.
strcmp(str1,str2)	Compare two strings str1 and str2.

All these functions are defined in **string.h header file**.

1 : strlen(string) – String Length : This function is used to count and return the number of characters present in a string.

Syntax : var=strlen(string);

Ex : Progrm using strlen() function

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```
    char name[]="JBREC";
```

```
    int len1,len2;
```

```
    clrscr();
```

```
    len1=strlen(name);
```

```
    len2=strlen("JBRECECE");
```

```
    printf("The string length of %s is: %d\n",name,len1);
```

```
    printf("The string length of %s is: %d","JBRECECE",len2);
```

```
    getch();
```

```
}
```

OUTPUT :

The string length of JBREC is : 5

The string length of JBRECECE is :8

Write a program to find the length of string

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```



```

char str[10];

int index;

printf("Enter the string:");

scanf("%s",str);

for(index=0;str[index]!=0;index++);

printf("The length of string is:%d",index);

getch();
}

```

OUTPUT :

Enter the string : subbareddy

The length of string is :10

2 : strcpy(string1,string2) – String Copy : This function is used to copy the contents of one string to another string.

Syntax : strcpy(string1,string2);

Where

string1 : is the destination string.

string 2: is the source string.

i.e the contents of string2 is assigned to the contents of string1.

Ex : Progrm using strcpy() function

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```

char str1[]="REDDY";
char str2[10];
strcpy(str2,str1);
printf("The string1 is :%s\n",str1);
printf("The string2 is :%s\n",str2);
strcpy(str2,str1+1);
printf("The string1 is :%s\n",str1);
printf("The string2 is :%s",str2);
}

```

OUTPUT :

The string1 is : REDDY

The string2 is : REDDY

The string1 is : REDDY

The string2 is : EDDY

//Write a program to copy contents of one string to another string.

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
    char str1[10],str2[20];
```

```
    int index;
```

```
    printf("Enter the string\n");
```

```
    scanf("%s",str1);
```

```
    for(index=0;str1[index]!='\0';index++)
```

```

        str2[index]=str1[index];
        str2[index]='\0';
        printf("String1 is :%s\n",str1);
        printf("String2 is :%s\n",str2);
        getch();
    }

```

OUTPUT :

Enter the string : cprogramming

String1 is : cprogramming

String2 is : cprogramming

3 : `strlwr(string)` – String LowerCase : This function is used to converts upper case letters of the string in to lower case letters.

Syntax : `strlwr(string);`

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```
    char str[]="JBREC";
```

```
    clrscr();
```

```
    strlwr(str);
```

```
    printf("The lowercase is :%s\n",str);
```

```
    getch();
```

```
}
```

OUTPUT : The lowercase is : jbreC

Write a program to which converts given string in to lowercase.

```
#include<stdio.h>
#include<conio.h>
void main()
{
    char str[10];
    int index;
    printf("Enter the string:");
    scanf("%s",str);
    for(index=0;str[index]!='\0';index++)
    {
        if(str[index]>='A' && str[index]<='Z')
            str[index]=str[index]+32;
    }
    printf("After conversion is :%s",str);
    getch();
}
```

OUTPUT : Enter the string : SUBBAREDDY

After conversion string is :subbareddy

4 :strupr(string) – String UpperCase : This function is used to converts lower case letters of the string in to upper case letters.

Syntax :strupr(string);

Program usingstrupr() function.

```
#include<stdio.h>

#include<conio.h>

#include<string.h>

void main()

{

    char str[]="jbrec";

   strupr(str);

    printf("UpperCase is :%s\n",str);

    getch();

}
```

OUTPUT : UpperCase is : JBREC

Write a program to which converts given string in to uppercase.

```
#include<stdio.h>

#include<conio.h>

void main()

{

    char str[10];

    int index;

    printf("Enter the string:");

    scanf("%s",str);

    for(index=0;str[index]!='\0';index++)

    {
```

```

    if(str[index]>='a' && str[index]<='z')
        str[index]=str[index]-32;
    }
    printf("After conversion is :%s",str);
    getch();
}

```

OUTPUT : Enter the string : subbareddy

After conversion string is :SUBBAREDDY

5 : strcmp(string1,string2) – String Comparison : This function is used to compare two strings to find out whether they are same or different. If two strings are compared character by character until the end of one of the string is reached. If the two strings are same strcmp() returns a value zero. If they are not equal, it returns the numeric difference between the first non-matching characters.

Syntax : strcmp(string1,string2);

Program using strcmp() function

```

#include<stdio.h>
#include<conio.h>
#include<string.h>
void main()
{
    char str1[]="reddy";
    char str2[]="reddy";
    int i,j,k;
    i=strcmp(str1,str2);
    j=strcmp(str1,"subba");
    k=strcmp(str2,"Subba");
    printf("%5d%5d%5d\n",i,j,k);
}

```

```
}
```

OUTPUT : 0 -1 32

Write a C program to find the comparison of two strings.

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
#include<string.h>
```

```
void main()
```

```
{
```

```
    char str1[10],str2[20];
```

```
    int index,l1,l2,flag=1;
```

```
    printf("Enter first string:");
```

```
    scanf("%s",str1);
```

```
    printf("Enter second string:");
```

```
    scanf("%s",str2);
```

```
    l1=strlen(str1);
```

```
    l2=strlen(str2);
```

```
    printf("Length of string1:%d\n",l1);
```

```
    printf("Length of string2:%d\n",l2);
```

```
    if(l1==l2)
```

```
    {
```

```
        for(index=0;str1[index]!='\0';index++)
```

```
        {
```

```
            if(str1[index]!=str2[index])
```

```
            {
```

```
                flag=0;
```

```

        break;
    }
}
}
else
    flag=0;
if(flag==1)
    printf("Strings are equal");
else
    printf("Strings are not equal");
}

```

OUTPUT : Enter the first string :jbrec

Enter the second string:jbrec

Length of string1 :5

Length of string2 :5

Strings are equal

6: strcat(string1,string2) – String Concatenation : This function is used to concatenate or combine, two strings together and forms a new concatenated string.

Syntax : strcat(sting1,string2);

Where

string1 : is the first string1.

string2 : is the second string2

when the above function is executed, string2 is combined with string1 and it removes the null character (\0) of string1 and places string2 from there.

Program using strcat() function.

```
#include<stdio.h>

#include<conio.h>

#include<string.h>

void main()

{

char str1[10]="jbrec";

char str2[]="ece";

strcat(str1,str2);

printf("%s\n",str1);

printf("%s\n",str2);

getch();

}
```

OUTPUT : jbrecece

ece

7 : strrev(string) - String Reverse :This function is used to reverse a string. This function takes only one argument and return one argument.

Syntax : strrev(string);

Ex : Program using strrev() function

```
#include<stdio.h>

#include<conio.h>

#include<string.h>

void main()

{
```